so it seems that the crashing issue we discussed in the "IFMC performance issues on Windows" chat is largely caused by a dying computer (hdmi connectivity problems, crashes and trouble rebooting, etc), a replacement is being ordered. that said, there still may be some underlying issues that trigger the problem that i would like to get to the bottom of...

my colleague sent the following analysis. can you please investigate/validate

Show more

1:12 PM

Examined colleague's CPU claims and planned codebase investigation strategy

I'll validate your colleague's claims against the actual code rather than taking them at face value — the core premise about `core.wait()` is checkable and I want to see what the codebase actually does.

Quick note on session protocol: since this is investigate-and-plan (not editing yet), I'll read the `/mnt/project/` copies to validate. But those can lag your local working copies, so **before we actually touch any file, push your current local copies** so we're not planning fixes against stale code.

Let me look at the real waits.

Excavated codebase patterns to validate performance claims

Let me check every `core.wait` in the codebase and whether `hogCPUperiod` is ever set:

Validated colleague's CPU claim and identified verification strategy

`hogCPUperiod` is never set anywhere, so every `core.wait()` uses PsychoPy's default. Let me confirm what that default actually does rather than trust my memory — checking whether PsychoPy source is available:

Scrutinizing core.wait() mechanics and sleep implementation details

PsychoPy isn't installed here, but I'm confident on the mechanism and want to state it precisely. Let me read the actual wait sites and their durations before doing the arithmetic:

Verified timing parameter and mapped investigation strategy

`post_stim_pause` is already `0.1`, not `0.5` — the colleague's "0.5s post-trial pause" claim is stale. Now the ITI value and the play_trial wait:

Validated colleague's CPU analysis, identified calculation errors, devised targeted optimization plan

Let me nail down the actual IFMC config count, since the colleague's "35 configs / 525–700 trials" drives his total-time figure:

Validated colleague's math, identified specific errors, and charted incremental fixes

I dug through the actual code. Your colleague is **directionally right but wrong on the mechanism**, and one of his key numbers is stale. Details below.

Validation of the claims

Claim	Verdict	Evidence
`core.wait()` spins CPU 100% **for its entire duration**	**False** — this is the central error	PsychoPy's `core.wait(secs, hogCPUperiod=0.2)` `time.sleep()`s for `secs - 0.2` and only busy-waits the **final 0.2s**. Codebase never overrides `hogCPUperiod` (grep: zero hits), so every wait uses this default.
Three `core.wait()`s per trial (2× ITI + play_trial)	**True**	`adaptive_paradigm.py:1404/1406` (iti_pre/iti_post), `trial_player.py:259` (play gate).
ITI ≈ 200–500 + 500–800 ms (up to 1.3s)	**Overstated**	`trial_ITI = 1.0` split into `iti_pre + iti_post` that **sum to exactly 1.0s** — not 1.3s.
`core.wait(0.5)` "avoid clicks" pause = 500ms	**Stale**	`post_stim_pause = 0.1` (line 202), already reduced. It's 100ms, and since 0.1 < 0.2 it's the *only* wait that fully hogs.
5 probe × 7 ratios = 35 configs, 525–700 trials	**True**	`IFMC_PROBE_FREQS`=5, `IFMC_MASKER_RATIOS`=7 → 35 (verified in `standard_frequencies.py`).
~1.73s CPU pinned per trial, 15–20 min **continuous** 100%	**False (~2.5× over)**	See corrected math below.
His fix for play_trial: `core.wait(secs, hogCPUperiod=0.2)`	**No-op**	That's already the default. Changing nothing there.

Corrected per-trial busy-wait

Only the hog tails actually spin:

- `iti_pre` (0.2–0.5s) → **0.2s** hog
- `iti_post` (0.5–0.8s) → **0.2s** hog
- `post_stim_pause` (0.1s) → **0.1s** (fully hogged, <0.2)
- `play_trial` gate (~0.42s incl. ~85ms ringout) → **0.2s** hog

≈ **0.7s/trial**, not 1.73s. And it's **intermittent** 100–200ms bursts separated by real sleep — with participant response time, that's a ~20–28% duty cycle on *one* core, not a continuous 15–20 min pin. Over 525–700 trials the fix removes ~370–490s of scattered spinning.

Bottom line: the fix is *correct and worth doing* (yielding the CPU is strictly better on a thermally-marginal box), but it is not the smoking gun he describes — and it's a secondary suspect behind the operator-display redraw hypothesis from the prior investigation (note: `show_operator_display` isn't even set in `test_ifmc.py` currently). On a machine that's already failing HDMI/reboots, I wouldn't expect this to be *the* cause.

Plan (minimal, incremental)

1. Replace the 3 non-critical waits with CPU-yielding sleeps:
   - `adaptive_paradigm.py:1404,1406` → `time.sleep(iti_pre/iti_post)`
   - `adaptive_paradigm.py:1878` → `time.sleep(self.post_stim_pause)`
2. `trial_player.py:259` play gate: audio start is sample-accurate via PTB `play(when=...)` — this wait is only a "don't-return-early" gate, so its precision is irrelevant. `time.sleep` never returns early (it overshoots, esp. ~15ms Windows granularity), so switching is safe and removes the last 0.2s hog. **Or** leave as-is (already only 0.2s). I lean toward switching for consistency.
3. Confirm nothing depends on `core.wait` side effects: it does **not** poll keys or pump the window, so no quit-handling/event-loop behavior is lost.
4. Import `time` where needed (`adaptive_paradigm.py` — verify it's imported).
5. Mac+Windows identical behavior; no timing-critical path touched.

This is a difficulty-2 change, so per your workflow it can skip a formal PRD — but I want the questions below answered first.

Unresolved questions

1. **Play gate (`trial_player.py:259`): switch to `time.sleep` or leave the 0.2s hog?** Switching is cleaner and safe; leaving is zero-risk. Your call.
2. **Route through a single `self._wait(secs)` base-class method** (fits your "drop-in updates" OO preference, one swap point later) **or** inline `time.sleep` at each site (smaller diff, matches "don't touch what we're not working on")? I'd slightly favor inline now given the change is tiny.
3. Given the dying machine + operator-display hypothesis, do you want this shipped now as cheap hygiene, or held until the replacement arrives so we can A/B whether it actually changes the thermal profile?
4. Confirm you'll push current local copies of `adaptive_paradigm.py` and `trial_player.py` before I generate edits — these two are exactly the stale-clobber-prone paradigm-adjacent files.

